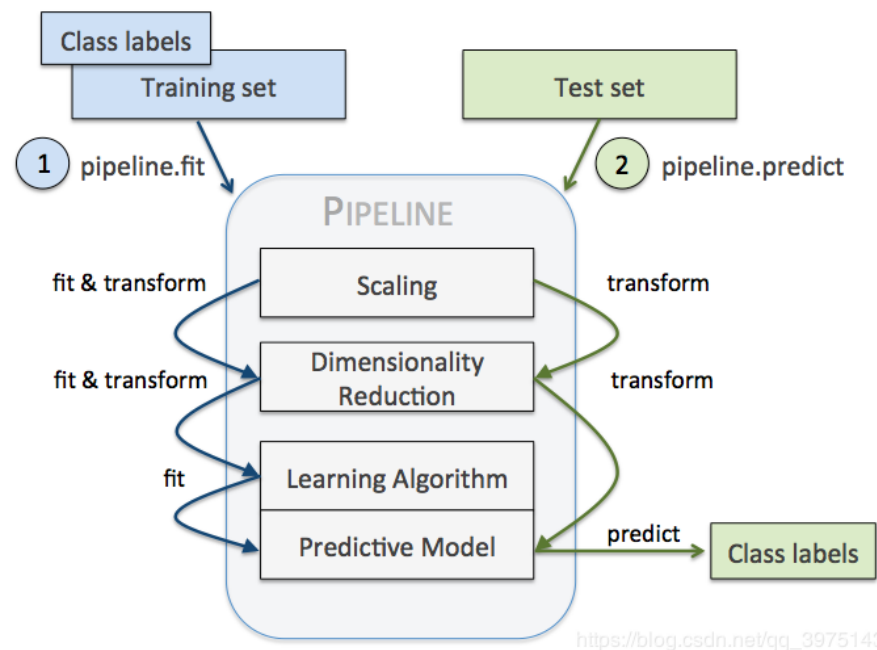


파이프 라인

• 파이프 라인

> 파이프 라인

- 데이터 전처리에서 학습, 예측까지의 과정을 하나로 연결해주는 기능
- `from sklearn.pipeline import Pipeline`



- 파이프 라인

- > 파이프 라인

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
```

```
data = load_iris()
X, y = data.data, data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- 파이프 라인

- > 파이프 라인

```
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])

pipeline.fit(X_train, y_train)
accuracy = pipeline.score(X_test, y_test)
print(f"테스트 정확도: {accuracy:.4f}")
```

Out : 테스트 정확도: 1.0000

- 파이프 라인

05

> 파이프 라인

```
pipeline.named_steps['scaler']
```

Out :

```
▼ StandardScaler  
StandardScaler()
```

```
pipeline.named_steps['model']
```

Out :

```
▼ LogisticRegression  
LogisticRegression()
```

- 파이프 라인

- > 파이프 라인과 함께 쓰기 좋은 estimator 전처리

- SimpleImputer

- 사이킷런의 누락값 전처리 모델
 - `from sklearn.impute import SimpleImputer`

- ColumnTransformer

- 전처리를 컬럼별로 적용시키는 모델
 - `from sklearn.compose import ColumnTransformer`

- FunctionTransformer

- 특정 함수를 estimator로 변환시키는 모델
 - `from sklearn.preprocessing import FunctionTransformer`

- 파이프 라인

- > SimpleImputer

```
from sklearn.impute import SimpleImputer
```

```
titanic = pd.read_csv('data/titanic.csv')
```

```
simpleimputer = SimpleImputer()
```

```
simpleimputer.fit_transform(titanic[['Age']])
```

```
Out: array([[22.,      ],
           [38.,      ],
           [26.,      ],
           [35.,      ],
           [35.,      ],
           [29.69911765],
           [54.,      ],
           [ 2.,      ],
           [27.,      ],
           [14.,      ],
           [ 4.,      ]])
```

- 파이프 라인

08

- > SimpleImputer

```
import numpy as np

simpleimputer = SimpleImputer(missing_values=np.NaN,
                              strategy='constant',
                              fill_value='N',)

simpleimputer.fit_transform(titanic[['Cabin']])
```

Out : array([['N'],
 ['C85'],
 ['N'],
 ['C123'],
 ['N'],
 ['N'],
 ['E46'],
 ['N'],

- 파이프 라인

> SimpleImputer

```
titanic = pd.read_csv('data/titanic.csv')
```

```
X = titanic.drop(['Survived', 'PassengerId', 'Name', 'Ticket',  
                  'Cabin', 'Embarked', 'Sex'], axis = 1)
```

```
y = titanic['Survived']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state=0)
```

- 파이프 라인

- > SimpleImputer

```
pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])
```

```
pipeline.fit(X_train, y_train)
accuracy = pipeline.score(X_test, y_test)
print(f"테스트 정확도: {accuracy:.4f}")
```

Out : 테스트 정확도: 0.7374

- 파이프 라인

011

- > ColumnTransformer

```
from sklearn.compose import ColumnTransformer  
from sklearn.preprocessing import OneHotEncoder
```

```
X = titanic.drop(['Survived', 'PassengerId', 'Name', 'Ticket', 'Cabin'], axis = 1)  
y = titanic['Survived']
```

```
cat_features = ['Embarked', 'Sex']  
num_features = ['Age', 'Fare']
```

- 파이프 라인

012

- > ColumnTransformer

```
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), num_features),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_features)
], remainder='passthrough')
```

```
result = preprocessor.fit_transform(X)
print(result)
```

Out : array([[-0.53037664, -0.50244517, 0. , ..., 3. ,
 1. , 0.],
 [0.57183099, 0.78684529, 1. , ..., 1. ,
 1. , 0.],
 [-0.25482473, -0.48885426, 0. , ..., 3. ,
 0. , 0.],
 ...,

- 파이프 라인

013

> ColumnTransformer

```
preprocessor.get_feature_names_out()
```

Out: array(['num__Age', 'num__Fare', 'cat__Embarked_C', 'cat__Embarked_Q',
 'cat__Embarked_S', 'cat__Embarked_nan', 'cat__Sex_female',
 'cat__Sex_male', 'remainder__Pclass', 'remainder__SibSp',
 'remainder__Parch'], dtype=object)

- 파이프 라인

- > FunctionTransformer

```
from sklearn.preprocessing import FunctionTransformer
```

```
def extract_cabin_initial(X):
```

```
    X = X.copy()
```

```
    X['Cabin'] = X['Cabin'].fillna('N').astype(str).str[0]
```

```
    return X
```

```
cabin_transformer = FunctionTransformer(extract_cabin_initial, validate=False)
```

```
cabin_transformer.fit_transform(titanic[['Cabin']])
```

- 파이프 라인

015

- > 전체 파이프 라인 작업

```
import pandas as pd
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler, FunctionTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

titanic = pd.read_csv('data/titanic.csv')
```

- 파이프 라인

016

- > 전체 파이프 라인 작업

```
X = titanic.drop(['Survived', 'PassengerId', 'Name', 'Ticket'], axis=1)
y = titanic['Survived']
```

```
num_features = ['Age', 'Fare']
cat_features = ['Sex', 'Embarked', 'Cabin']
impute_features = ['Age', 'Embarked', 'Cabin']
```

```
def extract_cabin_initial(X):
    X = X.copy()
    X['Cabin'] = X['Cabin'].fillna('N').astype(str).str[0]
    return X
```


- 파이프 라인

017

- > 전체 파이프 라인 작업

```
cabin_transformer = FunctionTransformer(extract_cabin_initial, validate=False)

imputer = ColumnTransformer([
    ('impute_age', SimpleImputer(strategy='mean'), ['Age']),
    ('impute_cat', SimpleImputer(strategy='constant', fill_value='N'), ['Embarked', 'Cabin'])
], remainder='passthrough')

def to_dataframe(X_array):
    columns = impute_features + [col for col in X.columns if col not in impute_features]
    return pd.DataFrame(X_array, columns=columns)

df_transformer = FunctionTransformer(to_dataframe, validate=False)
```

- 파이프 라인

018

- > 전체 파이프 라인 작업

```
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), num_features),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_features)
])
```

```
pipeline = Pipeline([
    ('cabin_extract', cabin_transformer),
    ('impute', imputer),
    ('to_dataframe', df_transformer),
    ('preprocess', preprocessor),
    ('model', LogisticRegression())
])
```

- 파이프 라인

019

- > 전체 파이프 라인 작업

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
pipeline.fit(X_train, y_train)
```

```
y_pred = pipeline.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f"테스트 데이터 정확도: {accuracy:.4f}")
```

Out : 테스트 데이터 정확도: 0.7821

- 파이프 라인

020

- > 파이프 라인 + 하이퍼 파라미터 튜닝

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, MinMaxScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.metrics import mean_squared_error

diabetes = load_diabetes()
X, y = diabetes.data, diabetes.target
```

- 파이프 라인

> 파이프 라인 + 하이퍼 파라미터 튜닝

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('poly', PolynomialFeatures()),
    ('model', Ridge())
])
```

- 파이프 라인

022

> 파이프 라인 + 하이퍼 파라미터 튜닝

```
param_grid = [  
    { "scaler" : [StandardScaler(), MinMaxScaler()],  
      "poly": [PolynomialFeatures(degree=2)],  
      "poly__degree" : [2,3],  
      "model": [Ridge(), Lasso()],  
      "model__alpha" : [0.01, 0.001]  
    },  
    { "scaler" : [StandardScaler(), MinMaxScaler()],  
      "poly": [None],  
      "model": [LinearRegression()],  
    }  
]
```

- 파이프 라인

- > 파이프 라인 + 하이퍼 파라미터 튜닝

```
grid_search = GridSearchCV(pipe, param_grid, cv=5, scoring='neg_mean_squared_error',
                             n_jobs=-1, verbose=2)

grid_search.fit(X_train, y_train)
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

print(f"최적의 모델: {grid_search.best_params_}")
print(f"테스트 데이터 MSE: {mse:.4f}")
```

Out : Fitting 5 folds for each of 18 candidates, totalling 90 fits
최적의 모델: {'model': LinearRegression(), 'poly': None, 'scaler': StandardScaler()}
테스트 데이터 MSE: 2900.1936